

ULC-ICAM

MON MIAM MIAM

Rapport technique et déroulement du projet

Groupe 2 • ZeDuc-Sp@ce • Kinshasa

MEMBRES DU GROUPE

Jean d'Arc Kazamba
Yveline Mwika
Christian Onokola
Wivine Mukendi
Aksa Mbala
Roseline Bolongosha
Caleb Kashoba

6 novembre 2026

Résumé

Mon Miam Miam est une plateforme de restauration multi-établissements destinée aux étudiants et aux équipes de restaurant. Elle réunit un site public, des espaces web par rôle, une application Expo pour étudiants et employés, une entrée bureau Tauri, une API Laravel et une base MySQL. Le projet est conduit selon une organisation inspirée d'Agile et de Scrum, avec Jira, GitHub, Figma et des contrôles automatisés. Le présent document décrit la conception, les quatre sprints, l'architecture et l'état du logiciel au 6 novembre 2026. Une fonctionnalité codée n'est considérée comme validée que lorsque son parcours a été vérifié.

1. Contexte et cahier des charges

Le cahier des charges V1.0.2 du 16 septembre 2024 répond à la dispersion des commandes par appel ou message, au suivi difficile des préparations, au manque de visibilité sur les plats disponibles et à la faible centralisation des ventes. Il prévoit la consultation du menu, le panier, la commande sur place ou en livraison, le traitement par les employés, la supervision par le gérant, l'administration, les réclamations et la fidélisation. La date du document source ne constitue pas le calendrier de développement de l'équipe en 2026.

L'équipe a étendu le périmètre vers plusieurs restaurants, un administrateur de plateforme distinct de l'administrateur local, une application mobile Expo, une entrée bureau Tauri, les contenus promotionnels et les jeux par établissement. ZeDuc-Sp@ce désigne la plateforme ; Khalifa_food et sleepy hollow servent de restaurants de recette. Les prix sont exprimés en francs congolais entiers (CDF) et les parcours sont situés à Kinshasa.

2. Organisation de l'équipe

Pendant la conception, Jean d'Arc Kazamba a travaillé sur le dépôt Git et les conventions de collaboration ; Roseline Bolongosha sur le suivi Jira ; Christian Onokola sur les maquettes ; Caleb Kashoba sur les données ; Aksa Mbala, Wivine Mukendi et Yveline Mwika sur les contrats API. Pendant le développement, les domaines principaux ont été répartis entre administration et intégration (Jean d'Arc), web étudiant (Christian), espaces employé et gérant et coordination (Roseline), authentification et droits (Aksa), menus, paniers et commandes (Wivine), modèles et migrations (Caleb), fidélité et promotions (Yveline). Il s'agit d'une répartition de responsabilités, non d'une certification individuelle de chaque ligne de code.

La formalisation Scrum proposait Jean d'Arc comme Product Owner et Roseline comme Scrum Master. La tenue effective des événements Scrum et des estimations collectives doit être appuyée par les comptes rendus correspondants. Les User Stories sont regroupées dans l'Epic Jira PMEE-46 ; les activités d'organisation figurent notamment de PMEE-54 à PMEE-59.

3. Déroulement des quatre sprints



Sprint 1 - analyse et conception. L'équipe a étudié le cahier des charges, identifié les rôles, décrit les parcours, préparé le backlog et les maquettes, établi les conventions Git et repéré les dépendances entre interface, API et données. Les livrables sont principalement documentaires.

Sprint 2 - socle technique. React a servi de base aux interfaces web ; Laravel a structuré routes, contrôleurs et modèles ; la base relationnelle, l'authentification et les permissions ont été préparées. Les procédures Jira-GitHub et la CI frontend ont été documentées. La fusion d'un socle ne prouve pas à elle seule les parcours métier.

Sprint 3 - fonctionnalités métier. Les espaces étudiant, employé, gérant, admin restaurant et admin plateforme, le site public et Expo ont été développés. L'API couvre les restaurants et leurs contenus, panier, devis, commandes, transitions, réclamations, fidélité, parrainage, jeux, paiements préparatoires, notifications et statistiques. Les interfaces connectées relisent les données de l'API. Les contributions ont été intégrées progressivement ; une pull request fermée sans fusion ne doit pas être comptée comme intégrée.

Sprint 4 - intégration, qualité et documentation. La consolidation relie les clients web et mobiles à Laravel/MySQL, vérifie les droits par rôle et par restaurant, prépare les scripts Docker, le démarrage local et le rapport technique. L'audit des branches, la recette entre applications, les sauvegardes, la configuration de production et les médias réels restent des éléments à prouver par des résultats de test. La présente version constitue le livrable documentaire consolidé.

4. Architecture et circulation des données



Le site et les espaces web utilisent React/Vite ; le mobile utilise React Native et Expo SDK 58 ; Tauri présente une entrée bureau et charge les espaces professionnels. Les clients appellent l'API Laravel avec un jeton Bearer. Laravel vérifie le compte, le rôle et l'affectation au restaurant ; MySQL conserve les données. Une file Laravel sert aux notifications et aux exports. Le catalogue public ne livre que les contenus publiés, actifs et non expirés.

Une commande appartient à un seul restaurant. Laravel recalcule les prix, disponibilités, horaires et frais, conserve un instantané de chaque ligne et protège la création par Idempotency-Key. Les changements de statut portent une version attendue et un événement daté. L'étudiant, l'employé et les responsables relisent régulièrement l'API : le projet utilise un polling sécurisé, avec un délai de quelques secondes, et non encore un canal WebSocket.

5. Rôles, données et sécurité

L'étudiant crée lui-même son compte et consulte ses commandes, adresses, réclamations et avantages. L'employé traite les commandes et les disponibilités de son établissement. Le gérant supervise l'équipe, les livraisons, les tables et les validations. L'administrateur local configure son restaurant et rend les décisions locales. L'administrateur de plateforme crée et suspend les établissements, attribue les administrateurs locaux, voit l'audit et des agrégats limités. Il ne reçoit pas de commandes nominatives, de détails de paiement, d'adresses ou de motifs de réclamation ordinaire. Une réclamation escaladée ne lui expose que l'identifiant, le restaurant, le statut et la date.

Les sessions web et mobiles sont restaurées après retour dans l'application ; la déconnexion révoque le jeton lorsque l'API répond et efface la copie locale. Le site public demeure consultable pendant la connexion et propose « Mon espace ». Les données privées sont isolées par restaurant côté serveur, au-delà du filtrage d'interface.

6. Modèle de données et contrats

Le schéma inclut utilisateurs, restaurants et affectations, catégories et contenus, variantes et suppléments, créneaux et tables, paniers, commandes et événements, adresses et positions de livraison, réclamations et pièces jointes privées, règles et mouvements de fidélité, parrainages, participations aux jeux, intentions et événements de paiement, et notifications. Les validations répondent de manière structurée ; les versions obsolètes renvoient HTTP 409. Les jetons et clés de paiement restent côté serveur.

Le paiement réellement préparé pour la recette est l'espèce à régler (cash_due). Visa, Mastercard, mobile money et crypto sont des options d'interface ou des adaptateurs à valider avec des contrats et sandboxes ; ils ne constituent pas

des encaissements opérationnels. Seul un webhook signé et vérifié peut confirmer paid dans la préparation serveur. Les règles de points doivent être validées métier avant activation ; une participation à un jeu n'atteste pas un gain.

7. Vérification, difficultés et limites

Le dépôt comprend des tests Feature Laravel pour l'authentification, l'isolation des restaurants, les transitions, la confidentialité, les paiements et le parcours transversal étudiant-employé-gérant-plateforme. Les documents de recette détaillent les scénarios manuels. Les builds et le démarrage local dépendent aussi de Docker Hub, des dépendances Node et du réseau du poste. Au 6 novembre 2026, un build Docker local a révélé une dépendance SQLite manquante puis un délai TLS vers Docker Hub ; la dépendance et la taille du contexte Docker ont été corrigées, mais une reconstruction complète réussie doit encore être constatée.

Les risques restants concernent la validation visuelle sur téléphone et bureau, la latence de polling, la montée en charge, les médias publiés, les paiements externes, les règles de remise et de gain, la sauvegarde/restauration MySQL, HTTPS et CORS en hébergement, ainsi que la tenue de la recette finale. L'inscription aux événements et l'encaissement électronique ne doivent pas être présentés comme disponibles tant que leurs intégrations ne sont pas validées.

8. Gestion et traçabilité

Jira relie les besoins à des tickets et User Stories ; GitHub conserve branches, commits, issues et pull requests ; Figma porte les maquettes ; GitHub Actions exécute des contrôles selon les workflows du dépôt. Les responsabilités ont évolué entre conception et réalisation. L'audit des pull requests distingue les contributions fusionnées des contributions fermées. La clôture administrative d'un ticket reste distincte d'un résultat de recette fonctionnelle.

9. Conclusion

Le groupe 2 a constitué un écosystème cohérent autour d'une API commune et de frontières de droits adaptées à plusieurs restaurants. L'organisation par sprints et la documentation ont permis de suivre la progression depuis le cahier des charges jusqu'à l'intégration. Le passage à une livraison exploitable exige encore les preuves de recette de bout en bout, les contenus définitifs et la configuration sécurisée de production.

Annexe A. Parcours de recette entre applications

Le scénario principal démarre par la publication d'un plat dans l'administration du restaurant A. Le site public et les clients étudiants doivent voir ce plat uniquement dans A. L'étudiant choisit A, ajoute le plat au panier, demande un devis et confirme une commande. Le serveur renvoie un identifiant ; la répétition de la même clé d'idempotence ne crée aucune deuxième commande. Un employé affecté à A reçoit cette commande après synchronisation et la fait progresser de « reçue » à « acceptée », « en préparation », « prête » puis « servie » ou « livrée » selon le service. L'étudiant voit le statut mis à jour ; le gérant et l'admin local voient les opérations de A ; la plateforme ne voit que des volumes agrégés. Un employé de B et l'admin plateforme ne peuvent consulter ni le nom ni l'adresse de cet étudiant.

Les cas de refus doivent être testés séparément : compte suspendu, jeton expiré, affectation retirée, panier mélangeant deux restaurants, rupture de stock, changement de version concurrent, saut de statut, adresse d'un autre étudiant, réclamation ordinaire consultée par la plateforme et webhook de paiement sans signature valide. Les réponses attendues sont notamment 401 pour l'absence de connexion, 403 pour les droits, 404 pour les ressources non visibles, 409 pour la concurrence et 422 pour une saisie invalide. Ces tests ne remplacent pas la vérification sur deux téléphones et navigateurs distincts.

Annexe B. Exploitation et lancement local

Le dépôt contient un Compose pour MySQL, Laravel, la file et le frontend ; Expo et Tauri sont ouverts par les scripts Windows. Le lanceur du Bureau détecte l'adresse réseau du PC pour Expo Go. Les variables .env contiennent les paramètres de base, la clé Laravel et l'URL API. Le démarrage vérifie la disponibilité de l'API et du web ; la migration prépare les tables. L'arrêt des conteneurs ne supprime pas le volume MySQL. Les sauvegardes, restaurations, certificats et secrets doivent être vérifiés sur l'environnement cible avant publication. Un téléphone sur partage de connexion iPhone peut nécessiter l'autorisation du port Metro dans le pare-feu Windows.

Références techniques et outils de gestion

- Dépôt GitHub : Projet_DeveloppementWeb
- Jira PMEE : PMEE
- Cahier des charges : CAHIER_DES_CHARGES.md
- Backlog : BACKLOG.md
- Répartition de conception : REPARTITION_PHASE_CONCEPTION.md
- Passage à l'implémentation : PASSAGE_CONCEPTION_IMPLEMENTATION.md
- Architecture : ARCHITECTURE.md
- Design system : DESIGN_SYSTEM.md
- Plan de tests : PLAN_DE_TESTS.md
- Risques : RISQUES.md
- Guide Jira, GitHub et CI : GUIDE_JIRA_GITHUB_CI.md
- Sprint 4 : document-technique
- Audit PR : AUDIT_PULL_REQUESTS_ET_INTEGRATION.md
- Checklist de recette : CHECKLIST_RECETTE.md
- Issues 85 et 86 : 85 et 86
- PR fusionnées signalées : 48 , 59 , 77 , 81 , 82 , 80 et 87 . PR fermées sans fusion signalées : #60, #79, #83, #84.
- Epic Jira : PMEE-46

Note de suivi Jira transmise pour le 6 novembre 2026 : 57 tickets, dont 45 « Terminé » et 12 « À faire ». Ce décompte est à reconfirmer dans Jira avant soutenance.